セルフオーダーシステム 制作⑤　店舗側の注文一覧

今回作るもの：お客様の注文を店員が確認できる「店舗 注文一覧」画面

目標　/Staff を開くと、注文日時・テーブル番号・商品名・数量・合計・注文状態が表示される

完成画面のイメージ

注文日時	卓番	注文商品	合計	状態
11/10 14:20	3	ハンバーグ ×2	2,600 円	新規注文
11/10 14:15	5	カレー ×1	980 円	新規注文

※ 金額と商品名は表示例です。実際には SQL Server の注文データを表示します

注文が表示されるまでの流れ

お客様のスマホ　|　注文確定 → Orders／OrderItems に保存

↓

SQL Server LocalDB　|　同じ注文データを保持

↓

店舗 PC のブラウザ　|　GET /Staff を送信

↓

StaffController → Razor　|　DB から取得 → HTML の注文一覧を表示

今回の範囲：注文を「表示する」まで。状態変更や自動更新は次回以降。

P2　STEP 1　制作④を master へマージする

まず制作④までの完成状態を master に保存し、制作⑤用のブランチを作ります

順番	Visual Studio での操作
1	制作④の作業ブランチで［Git の変更］を開き、未コミットの変更をコミット
2	［Git リポジトリ］で master に切り替える
3	制作④のブランチを master へマージする
4	master でビルドし、商品追加 → 注文確定まで確認する
5	master から staff-orders ブランチを新規作成して切り替える

注意：マージはソースコードの統合です。LocalDB 内の注文履歴がブランチごとに切り替わるわけではありません

今回作成するファイル

ファイル	操作
Controllers/StaffController.cs	新規作成：注文を DB から取り出す
Views/Staff/Index.cshtml	新規作成：一覧表として表示する

既存の Order・OrderItem・OrderStatus・OrderService・Program.cs・画像・CSS・DB 構造は変更しません。Migration も不要です

P3　STEP 2　Controller と DB の関係を確認する

今回の画面は、制作③で保存した注文データを「読み取る」だけです

URL：/Staff　|　ブラウザが GET リクエストを送信

↓

StaffController.Index()　|　EF Core で注文一覧を取得

↓

return View(orders)　|　一覧データを Razor へ渡す

↓

Views/Staff/Index.cshtml　|　繰り返し処理で表を作る

DB にある注文の構造

Orders（注文全体）	OrderItems（注文した商品）
Id・TableNumber・OrderedAtUtc・Status	ProductId・Quantity・UnitPrice
1 件の注文に対して	複数の注文明細が紐づく

EF Core の内容

コード	役割
Include(o => o.Items)	各注文の明細も取得
ThenInclude(i => i.Product)	明細の商品名を取得
AsNoTracking()	表示だけなので変更を追跡しない
OrderByDescending(...)	新しい注文から表示
Take(50)	最大 50 件に制限

重要：UnitPrice は注文時の単価です。後日商品価格が変わっても、過去の注文金額は変えません

P4　STEP 3　StaffController を新規作成する

対象：Controllers/StaffController.cs　【新規作成】

Controllers フォルダーを右クリック → ［追加］→［クラス］→ StaffController.cs。以下の全コードを入力します

```
using Microsoft.AspNetCore.Mvc;      // Controller を使用
using Microsoft.EntityFrameworkCore; // EF Core の非同期処理を使用
using SelfOrderSystem.Data;          // AppDbContext を使用

namespace SelfOrderSystem.Controllers;

public class StaffController : Controller
{
    private readonly AppDbContext _db; // DB への接続を担当

    public StaffController(AppDbContext db)
    {
        _db = db; // DI で受け取ったものを保存
    }

    [HttpGet] // 注文一覧を表示する GET リクエスト
    public async Task<IActionResult> Index()
    {
        var orders = await _db.Orders
            .AsNoTracking()                 // 表示専用なので追跡しない
            .Include(o => o.Items)          // 注文明細を一緒に取得
            .ThenInclude(i => i.Product)    // 明細に対応する商品を取得
            .OrderByDescending(o => o.OrderedAtUtc) // 新しい順
            .Take(50)                      // 最大 50 件
            .ToListAsync();                // 注文一覧を取得

        return View(orders); // Views/Staff/Index.cshtml に渡す
    }
}
```

チェック：namespace が SelfOrderSystem.Controllers になっているか、Include と ThenInclude の両方があるかを確認

P5　STEP 4　注文一覧の View を作る（前半）

対象：Views/Staff/Index.cshtml　【新規作成】

Views の中に Staff フォルダーを作り、Index.cshtml を追加します。P5 と P6 は 1 つのファイルにつなげて入力します

```
@using SelfOrderSystem.Models
@model List<Order>  @* 注文一覧を受け取る *@
@{
    ViewData["Title"] = "店舗 注文一覧";
}

<h1 class="h3">店舗 注文一覧</h1>
<p>注文日時の新しい順に最大 50 件を表示します</p>

@* 注文が 0 件なら案内だけを表示 *@
@if (Model.Count == 0)
{
    <div class="alert alert-info">まだ注文はありません</div>
}
else
{
    @* 幅の狭い画面では表を横スクロール *@
    <div class="table-responsive">
        <table class="table table-striped table-bordered align-middle">
            <thead>
                <tr>
                    <th>注文 ID</th>
                    <th>注文日時</th>
                    <th>卓番</th>
                    <th>商品と数量</th>
                    <th>合計</th>
                    <th>状態</th>
                </tr>
            </thead>
```

このページだけでは Razor の閉じ括弧が足りません。続けて P6 を入力してからビルドします

P6　STEP 4　注文一覧の View を作る（後半）

対象：Views/Staff/Index.cshtml　【P5 の直後に続ける】

```
            <tbody>
            @* 外側の繰り返しは注文を 1 件ずつ表示 *@
            @foreach (var order in Model)
            {
                <tr>
                    <td>@order.Id</td>
                    <td>@order.OrderedAtUtc.ToLocalTime().ToString("yyyy/MM/dd HH:mm")</td>
                    <td>@order.TableNumber</td>
                    <td>
                        @* 内側の繰り返しは各注文の商品を表示 *@
                        @foreach (var item in order.Items)
                        {
                            <div>@item.Product?.Name × @item.Quantity</div>
                        }
                    </td>
                    @* 注文した時点の単価で合計を計算 *@
                    <td>@order.Items.Sum(i => i.UnitPrice * i.Quantity).ToString("N0") 円</td>
                    <td>@order.Status.ToJapanese()</td>
                </tr>
            }
            </tbody>
        </table>
    </div>
}
```

2 重の foreach を図で理解

外側 foreach（注文 1 件）　|　卓番 3・11/10 14:20

↓

内側 foreach（注文の明細）　|　ハンバーグ ×2 ／ カレー ×1

↓

次の注文へ　|　卓番 5・11/10 14:15

@* … *@ は Razor のコメントです。HTML を含む.cshtml では C#の // コメントをそのまま書かないようにします

P7　STEP 5　実行・動作確認・コミット

ブラウザから実行する

F5 で SelfOrderSystem 起動　|　これまでのメニュー画面も表示できるか確認

↓

アドレス末尾に /Staff　|　店舗 注文一覧を表示

↓

別タブで注文を確定　|　カートから商品を注文

↓

店舗画面を再読み込み　|　新しい注文が先頭に増える

確認表

確認項目	正しい動作
注文が 0 件	「まだ注文はありません」が表示
注文を 2 回実行	新しい注文が上に表示
複数商品を注文	1 件の注文内に商品名と数量が複数行
注文合計	UnitPrice × Quantity の合計
既存の注文確定・注文状況	引き続き正常に動作

エラー時の確認

症状	確認する場所
/Staff で 404	StaffController の名前・Index()
View が見つからない	Views/Staff/Index.cshtml
商品名が表示されない	ThenInclude(i => i.Product)
ToJapanese()が見つからない	@using SelfOrderSystem.Models
時刻がずれる	ToLocalTime()は実行 PC の時刻設定に依存

最後に staff-orders へコミット　例：★制作 05 店舗側の注文一覧を表示

次回：注文の状態（新規注文 → 調理中 → 提供済み）を店舗側から変更します。